

Western Shooter

Advanced Technical Report and Study Guide
Interactive Graphics Course – Sapienza University of Rome

Flavio Massaroni (1990975)

Lorenzo Ugolini (1958654)

June 2026

Abstract

This report documents the architectural design, mathematical foundations, and real-time rendering optimizations of **Western Shooter**, a 3D interactive graphics application built using vanilla JavaScript and the WebGL-based library **Three.js** [1, 2]. Developed in accordance with the curriculum requirements of the Interactive Graphics course, this project avoids pre-recorded skeletal animations or external physics engines, relying instead on manual procedural animations, custom spatial hazard partitioning, real-time dynamic light culling, and rigorous optimization strategies to ensure a consistent 60 FPS performance envelope within a browser runtime environment [2].

Contents

1 Introduction and Academic Course Alignment

Western Shooter is a third-person action game set in a stylized frontier town in the American Old West [1]. The player controls a cowboy character navigating a desert environment populated by hostile robots that patrol and chase the player upon detection [1]. The core objective is to locate and eliminate all active robots using a revolver [1].

The technical implementation maps directly onto the rigorous curriculum guidelines established by the Interactive Graphics syllabus:

- **Complex Hierarchical Models:** Both the player character and the robot enemies utilize intricate skeletal bone hierarchies, which are dynamically manipulated in real-time [2].
- **Advanced Materials and Shading:** The environment features physically based materials (PBR) mapped with color, normal, roughness, and metallic textures [2, 3]. High Dynamic Range (HDR) images are utilized for environmental light sources [2].
- **User Interaction and Customization:** The application supports runtime parameter configurations, including difficulty adjustment, UI visibility toggles, standard and silver-mirror bullet choices, and a real-time day/night cycle [1, 2].
- **Procedural Animations:** In compliance with the strict prohibition against pre-baked animation files (“*YOU CANNOT IMPORT ANIMATIONS*”), every character pose, gait, vegetative wind sway, impact reaction, and death collapse is simulated programmatically on the CPU [2].

1.1 Librarian Abstraction: Why Three.js

While writing raw WebGL provides exposure to GPU buffers, writing boilerplate shader programs and manually managing draw calls diverts effort away from core graphic engineering topics [1]. Three.js is utilized as an abstraction layer to bypass low-level GPU setup, while providing mathematical constructs (quaternions, transformation matrices, ray-box intersections) necessary to build the application logic [1]. This abstraction allows the project to focus on high-performance procedural animation, spatial coordinate mapping, and runtime optimizations [2].

2 The WebGL Graphics Pipeline and Coordinate Transformations

Understanding how 3D assets transition from raw disk files to colored pixels on an HTML5 canvas requires analyzing the WebGL pipeline and the linear algebra governing coordinate systems [1, 2].

2.1 The WebGL Rendering Pipeline

The rendering pipeline operates in sequential stages:

1. **Application Stage (CPU)**: Executed in JavaScript. Generates or loads geometry, processes skeletal bone animations, updates coordinate matrices, and sends vertex streams (positions, normals, UVs) to GPU memory via WebGL buffer bindings.
2. **Vertex Shader Stage (GPU)**: A programmable GLSL stage that processes each vertex. It computes lighting vectors, applies skeletal weights, and maps local coordinates to Clip Space via matrix multiplication [1].
3. **Primitive Assembly & Rasterization (GPU)**: Vertices are grouped into triangles. The GPU discards geometry outside the viewing frustum (clipping) and interpolates vertex properties (normals, UVs) across the scanline to generate candidate fragments (pixels).
4. **Fragment Shader Stage (GPU)**: Computes the final color of each fragment [1]. This is where the physically based rendering (PBR) equations are calculated, combining surface albedo, metallic-roughness, normal maps, and environmental reflections [2, 3].
5. **Output Merger (GPU)**: Performs depth-testing (discarding occluded pixels), color blending, and writes the final pixel values to the WebGL framebuffer for display.

2.2 The Model-View-Projection (MVP) Transformation

A crucial aspect of computer graphics is mapping 3D local coordinates to 2D screen space [1]. This is achieved using the **Model-View-Projection (MVP)** matrix chain [1]. Let $\mathbf{x}_{\text{local}} = [x \ y \ z \ 1]^T$ be a vertex represented in homogeneous coordinates:

$$\mathbf{x}_{\text{clip}} = \mathbf{P} \cdot \mathbf{V} \cdot \mathbf{M} \cdot \mathbf{x}_{\text{local}} \quad (1)$$

2.2.1 1. Model Matrix (M)

Transforms local model space coordinates into World Space coordinates. It is constructed by multiplying individual scaling (**S**), rotation (**R**), and translation (**T**) matrices:

$$\mathbf{M} = \mathbf{T} \cdot \mathbf{R} \cdot \mathbf{S} \quad (2)$$

2.2.2 2. View Matrix (V)

Transforms World Space coordinates into Camera (Eye) Space, positioning the camera at the origin looking down the negative Z -axis. The View Matrix is computed as the inverse of the camera's world transformation matrix (**C**):

$$\mathbf{V} = \mathbf{C}^{-1} = \begin{bmatrix} \mathbf{R}_{\text{cam}}^T & -\mathbf{R}_{\text{cam}}^T \mathbf{t}_{\text{cam}} \\ \mathbf{0}^T & 1 \end{bmatrix} \quad (3)$$

2.2.3 3. Perspective Projection Matrix ($\mathbf{P}$)

Transforms Eye Space into Clip Space, simulating perspective projection where objects further from the camera appear smaller. This matrix scales X and Y based on the field of view (θ) and aspect ratio (a), and remaps Z into a non-linear depth value between near plane (n) and far plane (f):

$$\mathbf{P} = \begin{bmatrix} \frac{1}{a \cdot \tan(\theta/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\theta/2)} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad (4)$$

After multiplying by $\mathbf{P}$, the coordinates are in Clip Space. The GPU then divides the coordinates by the homogeneous coordinate w_c (which stores the value of $-z_{\text{eye}}$), performing the **perspective division**:

$$\mathbf{x}_{\text{ndc}} = \begin{bmatrix} x_{\text{clip}}/w_c \\ y_{\text{clip}}/w_c \\ z_{\text{clip}}/w_c \end{bmatrix} \quad (5)$$

This yields Normalized Device Coordinates (NDC) in the range $[-1, 1]^3$, which are mapped to screen viewport pixels.

3 Software Architecture and Game Loop Life Cycle

The application structure is organized using Object-Oriented Programming (OOP) paradigms, separating logical domains into dedicated classes: `Player`, `Bot`, `Bullet`, `Gun`, and `World`, coordinated by the core render loop in `main.js` [2, 3].

3.1 Asynchronous Asset Loading & Progress Tracker

Web-based graphic applications must ensure asset synchronization before starting the render loop [2]. If rendering begins before glTF meshes or HDR textures finish loading, null-pointer references can crash the loop [1, 2]. In `world.js`, a unified asynchronous tracking system resolves this issue:

```
1 _begin() {  
2   this._pending++;  
3   this._total++;  
4   this._updateBar();  
5 }  
6  
7 _done() {  
8   this._pending = Math.max(0, this._pending - 1);  
9   this._updateBar();  
10  if (this._pending === 0) this._finish();  
11 }
```

Listing 1: Asynchronous tracking algorithm in `world.js`

Every loader call triggers `_begin()`, incrementing active network requests. Upon success or error, `_done()` decrements the pending tracker. Only when pending requests reach zero is the `_finish()` callback executed. This dismisses the Loading Screen, initializes the spatial partitioning grid, and unlocks player controls.

3.2 The Bullet-Time Kill Camera Mechanics

When a bullet is predicted to hit an active target, the standard third-person orbit camera switches to the `killCamActive` state [2]. The camera detaches from the player and anchors to the flying projectile’s coordinate space [2, 5].

To emphasize the trajectory, the system applies two distinct time scales to the simulation:

- `botTimeScale = 0.01`: Freezes bot logical states, pathfinding, and joint animations [2, 5]. This prevents the player from taking damage or being attacked while unable to move during the non-interactive cinematic sequence [2].
- `bulletTimeScale = 0.15`: Limits the bullet travel speed to 15% of its normal rate [2, 5]. This slow-motion effect builds dramatic suspense and allows the viewer to observe the real-time environmental reflections mapping across the bullet’s metal surface [2, 6].

4 Geometry, Materials, and Image-Based Lighting

4.1 High Dynamic Range (HDR) Workflow and ACES Filmic Tone Mapping

Traditional image formats limit light intensities to the Standard Dynamic Range (SDR) range $[0, 1]$ (or 8 bits per channel). To simulate natural, varying light intensities, *Western Shooter* utilizes high-precision High Dynamic Range (HDR) images [2].

An `RGBELoader` parses the Radiance `.hdr` format, which stores floating-point data using the shared-exponent RGBE format [2]. These environmental maps serve two purposes:

1. **Skybox Rendering:** Rendered as the background to depict realistic sun and night skies [2, 3].
2. **Image-Based Lighting (IBL):** The HDR pixels are used as mathematical light probes [2, 3]. When rendering PBR materials, the ambient diffuse and specular reflections are evaluated against these environment maps, producing physically accurate ambient reflections [2].

Since standard displays cannot output raw HDR values $[0, \infty)$, they must be compressed into $[0, 1]$ using a Tone Mapping algorithm [1, 3]. This project implements **ACES Filmic Tone Mapping** (`THREE.ACESFilmicToneMapping`), which approximates the exposure curve of classic cinematic film [1, 2]:

$$f(x) = \frac{x(ax + b)}{x(cx + d) + e} \quad (6)$$

The coefficients ($a = 2.51, b = 0.03, c = 2.43, d = 0.59, e = 0.14$) are empirical parameters calibrated by the Academy of Motion Picture Arts and Sciences (AMPAS) to closely mimic the sensitometric exposure curves of physical, chemical film stock. Unlike simple linear clamping, ACES preserves details in high-intensity highlights (such as street lamp glow or intense sunset highlights) while maintaining contrast in midtones and dark areas [2].

4.2 Real-Time Environmental Reflections and Self-Occlusion Prevention

To show the ACES tone mapping curve in action on the silver mirror bullet option, dynamic environmental mapping is calculated in real-time [1]. A `CubeCamera` coupled with a `WebGLCubeRenderTarget` capturing 512×512 resolution maps the surroundings onto the bullet's surface [1, 5].

To prevent rendering errors, the bullet hides itself during reflection capture:

```
1 cubeCamera.position.copy(killCamBullet.mesh.position);
2 for (let i = 0; i < bullets.length; i++) {
3   if (bullets[i].mesh) bullets[i].mesh.visible = false;
4 }
5 cubeCamera.update(renderer, scene);
6 for (let i = 0; i < bullets.length; i++) {
7   if (bullets[i].mesh) bullets[i].mesh.visible = true;
8 }
```

Listing 2: Dynamic cube-map generation

If the bullet remained visible during `cubeCamera.update()`, the reflection pass would capture the interior of the bullet's own geometry [2, 6]. This results in self-occlusion, rendering the exterior reflective surface with distorted black shadows instead of environmental reflections [6].

5 Procedural Animation Mathematics

Every motion in the game is generated procedurally at runtime, ensuring complete compliance with course requirements prohibiting imported animations [2].

5.1 Joint Hierarchy Setup and the Rest Pose Baseline

To build joint movements, we first map the parent-child linkages of the loaded models [2]. When the GLTF assets are loaded into memory, their initial resting skeleton configuration (rest pose) must be cloned and cached before any transform logic is applied.

```
1 this.bones = {
2   hips: this.mesh.getObjectByName("mixamorigHips_01"),
3   spine: this.mesh.getObjectByName("mixamorigSpine_02"),
4   upperLegL: this.mesh.getObjectByName("mixamorigLeftUpLeg_047"),
5   upperLegR: this.mesh.getObjectByName("mixamorigRightUpLeg_052"),
6   lowerLegL: this.mesh.getObjectByName("mixamorigLeftLeg_048"),
7   lowerLegR: this.mesh.getObjectByName("mixamorigRightLeg_053"),
8   footL: this.mesh.getObjectByName("mixamorigLeftFoot_049"),
9   footR: this.mesh.getObjectByName("mixamorigRightFoot_054"),
10  upperArmL: this.mesh.getObjectByName("mixamorigLeftArm_08"),
11  upperArmR: this.mesh.getObjectByName("mixamorigRightArm_028"),
12 };
13
14 this.rest = {};
15 for (const [name, bone] of Object.entries(this.bones)) {
16   this.rest[name] = { quat: bone.quaternion.clone() };
17 }
```

Listing 3: Caching rest pose states on startup in player.js

Applying incremental rotative deltas ($\mathbf{q}_{\text{new}} = \mathbf{q}_{\text{prev}} \cdot \Delta \mathbf{q}$) directly to a bone's current quaternion inside a high-frequency render loop will rapidly cause floating-point drift due to machine rounding precision. This drift manifests as skeletal distortion, joint elongation, or progressive model tearing. To prevent this cumulative numeric deterioration, the cloned rest pose baseline ($\mathbf{q}_{\text{rest}}$) is preserved untouched. At every frame, we compute the total rotation relative to this clean baseline:

$$\mathbf{q}_{\text{joint}}(t) = \mathbf{q}_{\text{rest}} \cdot \mathbf{q}_{\text{offset}}(t) \quad (7)$$

5.2 The Walk Cycle Gait

The walk cycle uses coordinated periodic mathematical functions to animate thigh, knee, foot, and arm bones based on elapsed time t [2]:

$$\theta_{\text{thigh}}(t) = -\sin(\omega \cdot t) \cdot A_{\text{thigh}} \quad (8)$$

$$\theta_{\text{knee}}(t) = -\max(0, \cos(\omega \cdot t)) \cdot A_{\text{knee}} \quad (9)$$

The max function for θ_{knee} is critical: it models physical leg anatomy by preventing the joint from flexing forward (hyperextension), ensuring the joint only bends backward during the step cycle [2].

5.2.1 Hip Bobbing and Spine Counter-Rotation

To prevent the visual effect of feet sliding on the ground, the hip coordinate (**Hips**) undergoes vertical displacement (*Hip Bobbing*) calculated to match the compression of the legs during the

gait [2]:

$$y_{\text{hips}}(t) = y_{\text{base}} + (\cos(\theta_{\text{thigh}}) + \cos(\theta_{\text{thigh}} - \theta_{\text{knee}}) - 2) \cdot L_{\text{leg}} \quad (10)$$

Simultaneously, to maintain rotational momentum, the spine bone undergoes horizontal twisting (*Torso Counter-Rotation*) [2]:

$$\theta_{\text{spine}}(t) = -\sin(\omega \cdot t) \cdot A_{\text{spine}} \quad (11)$$

5.3 Mathematical Representation of Rotations: Quaternions vs. Euler Angles

In standard 3D programming, orientation can be defined using Euler angles (pitch, yaw, roll) [1]. However, Euler angles suffer from **Gimbal Lock**, a mathematical anomaly that occurs when two rotation axes align. This causes a loss of one rotational degree of freedom, resulting in sudden, erratic joint snapping.

To avoid this, all procedural bone rotations are calculated using **Quaternions** ($\mathbb{H}$). A quaternion represents a 3D orientation using a four-dimensional vector $\mathbf{q} = [w \ x \ y \ z]^T$ corresponding to a rotation angle θ around a unit axis $\mathbf{u} = [u_x \ u_y \ u_z]^T$:

$$\mathbf{q} = \begin{bmatrix} \cos(\theta/2) \\ u_x \sin(\theta/2) \\ u_y \sin(\theta/2) \\ u_z \sin(\theta/2) \end{bmatrix} \quad (12)$$

Quaternions prevent gimbal lock and enable smooth interpolation. Spherical Linear Interpolation (**SLERP**) interpolates smoothly between two orientations along the geodesic path on a 4D unit sphere:

$$\text{Slerp}(\mathbf{q}_0, \mathbf{q}_1, t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} \mathbf{q}_0 + \frac{\sin(t\Omega)}{\sin \Omega} \mathbf{q}_1 \quad (13)$$

This ensures smooth joints and constant angular velocities during procedural transitions, which is not possible with linear Euler interpolation.

5.4 Bot Collapse Death Animation

Upon projectile impact, a smoothstep easing function manages the transition from active to dead state over fixed window T [2, 5]:

$$\alpha = \frac{t_{\text{death}}}{T}, \quad s(\alpha) = \alpha^2 \cdot (3 - 2\alpha) \quad (14)$$

During this transition, the **Body** bone rotates backward by 90° ($-\pi/2$ rad) on the X axis, and limb bones flex outward to simulate joint collapse under gravity [2, 4].

6 Steering Behaviors and Finite State Machine AI

Active robots utilize a Finite State Machine (FSM) comprising two main behavioral states [2, 5]:

1. **Patrol State:** Robots wander along a randomized linear vector direction, which is periodically re-assigned at random intervals $T \in [2, 4]$ seconds [2, 5].
2. **Chase State:** If the distance to the player $d = \|\mathbf{p}_{\text{player}} - \mathbf{p}_{\text{bot}}\|$ falls below the detection radius ($R_{\text{chase}} = 12$), the robot computes a direction vector targeting the player [2, 5]:

$$\mathbf{d}_{\text{chase}} = \frac{\mathbf{p}_{\text{player}} - \mathbf{p}_{\text{bot}}}{\|\mathbf{p}_{\text{player}} - \mathbf{p}_{\text{bot}}\|} \quad (15)$$

The robot increases its travel speed from $V_{\text{patrol}} = 1.5$ to $V_{\text{chase}} = 2.5$ and rotates on its Y axis to directly face the player [2, 5].

6.1 Steering Separation Forces

To prevent robot models from clustering and overlapping while pursuing the player, the system applies a separation steering force. This repels robots when they get too close to one another [2, 5]:

```
1 if (otherBotPositions) {
2   for (const otherPos of otherBotPositions) {
3     Bot._vToOther.subVectors(this.mesh.position, otherPos).setY(0);
4     const toOther = Bot._vToOther;
5     if (toOther.length() < this.BOT_AVOID_RADIUS) {
6       this.moveDir.add(toOther.normalize().multiplyScalar(0.5)).normalize();
7     }
8   }
9 }
```

Listing 4: Separation vector calculations in bot.js

This flocking behavior ensures that multiple pursuers distribute themselves naturally across the playable space instead of merging into a single rendering entity [5].

7 Collision Detection and Performance Optimizations

Since collision testing runs in real-time, optimizing CPU performance is essential for maintaining smooth frame rates [2, 6].

7.1 Uniform 2D Spatial Hash Grid

Checking every moving entity against all static colliders in the environment at every frame creates an $\mathcal{O}(N \cdot M)$ complexity scaling bottleneck [2, 6]. To optimize this, *Western Shooter* implements a **Uniform 2D Spatial Hash Grid** [2, 6].

The playable environment is partitioned into grid cells of side length $L = 20$ [2]. During initialization, static bounding boxes (Box3) are mapped into any cell they overlap [2]:

```
1 _buildGrid() {
2   const CELL = 20;
3   const cells = new Map();
4   const key = (cx, cz) => cx + ',' + cz;
5
6   for (const box of this.colliders) {
7     const x0 = Math.floor(box.min.x / CELL);
8     const x1 = Math.floor(box.max.x / CELL);
9     const z0 = Math.floor(box.min.z / CELL);
10    const z1 = Math.floor(box.max.z / CELL);
11
12    for (let cx = x0; cx <= x1; cx++) {
13      for (let cz = z0; cz <= z1; cz++) {
14        const k = key(cx, cz);
15        if (!cells.has(k)) cells.set(k, []);
16        cells.get(k).push(box);
17      }
18    }
19  }
20  this._grid = { cells, CELL, key };
21 }
```

Listing 5: Building the grid partition

At runtime, collision detection for the player or active bullets computes the current cell key in a single step:

$$c_x = \lfloor \frac{x}{L} \rfloor, \quad c_z = \lfloor \frac{z}{L} \rfloor \quad (16)$$

By performing this integer mapping, the search query reduces from checking the entire global array down to an average $\mathcal{O}(1)$ lookup complexity, evaluating collisions only against the static objects inside that specific grid quadrant [2, 6].

7.2 Bullet Tunneling Prevention

At high velocities ($V = 60$), discrete frame-by-frame updates can cause a bullet to skip past static wall colliders or target hitboxes entirely without registering an impact [2, 5].

This tunneling effect is resolved using a two-pass detection approach:

1. **Broad-Phase:** Rapid AABB intersection checks verify overlapping bounding boxes [2, 5].
2. **Narrow-Phase:** A geometric line segment is projected from the bullet's position in the previous frame ($\mathbf{p}_{\text{prev}}$) to its current position ($\mathbf{p}_{\text{curr}}$) [2]. If this ray intersects a target bounding box within the travel distance, a hit is registered at the precise intersection point [2]:

```

1 _rayDir.subVectors(currPos, _prevPos).normalize();
2 _ray.set(_prevPos, _rayDir);
3 if (_ray.intersectBox(bot.hitbox, _intersectPoint)) {
4     if (_prevPos.distanceTo(_intersectPoint) <= distTraveled) {
5         rayHit = true;
6         exactHitPoint = _intersectPoint.clone();
7     }
8 }

```

Listing 6: Raycast intersection calculations

7.3 Memory Pooling (Zero Garbage Collection)

To prevent frame rate stuttering caused by JavaScript’s Garbage Collector (GC) cleaning up short-lived objects, the application uses memory pooling [2, 6]:

- **Particle Pooling:** Impact sparks and flash lights (`_particlePool` and `_flashPool`) are pre-allocated at startup [2]. When a bullet hits a target, active particles are retrieved from the pool, displayed, and returned upon expiration, avoiding allocation overhead [2].
- **Class-Level Variable Cache:** Mathematical operators within high-frequency loops are pre-allocated at the class level (e.g., `Bot._tmpQuat`) [2]. This prevents the creation of temporary instances on the heap, ensuring allocation-free loop cycles [2, 6].

8 Lighting, Shadows, and Dynamic Light Culling

The rendering environment alternates between a daytime desert scene and a nighttime scene [1, 2].

8.1 Nocturnal Point Light Shadow Culling

During the night phase, 20 street lamps provide local illumination using point lights [2, 3]. Unlike directional lights, shadow-casting point lights require rendering 6 depth cubemap passes per light to generate Omni-directional shadow maps [2, 6]. Doing this for all 20 lamps would require 120 shadow render passes per frame, which would severely degrade GPU performance [2, 6].

To keep rendering performance stable, a distance-based shadow culling algorithm is applied:

- The maximum active shadow-casting lights is capped at `MAX_SHADOW_LAMPS = 2` [2].
- At each frame, active lamps are sorted by their squared Euclidean distance to the player [2].
- The shadow-casting property is enabled (`castShadow = true`) only for the 2 nearest lamps, provided they are within 40 units of the player [2]. Shadows are disabled for the remaining 18 lamps [2].

This dynamic culling maintains high visual quality near the player while keeping the GPU workload light and stable across the scene [2].

9 Oral Examination Preparation Guide (Viva Voce)

Below is a compilation of theoretical and practical questions often asked during oral defenses in this course, accompanied by precise, technical answers.

9.1 Conceptual High-Yield Defense Questions

Question 1: Explain the MVP transformation. How are coordinates converted from Model Space to Clip Space, and what is the role of Homogeneous Coordinates?

Answer: The transformation follows the equation: $\mathbf{x}_{\text{clip}} = \mathbf{P} \cdot \mathbf{V} \cdot \mathbf{M} \cdot \mathbf{x}_{\text{local}}$. First, the Model Matrix ($\mathbf{M}$) transforms local coordinates to World Space. Next, the View Matrix ($\mathbf{V}$), which is the inverse of the camera's transformation matrix ($\mathbf{C}^{-1}$), maps coordinates from World Space to Eye Space. Finally, the Perspective Projection Matrix ($\mathbf{P}$) maps coordinates into Clip Space. Homogeneous coordinates (x, y, z, w) are used because 3D translation cannot be expressed as a linear 3×3 matrix multiplication. Using 4×4 matrices allows translation, rotation, and scaling to be combined into a single transform chain. Additionally, homogeneous coordinates are essential for perspective projection: the perspective matrix stores the negative Z coordinate in the w component, allowing the GPU to divide by w during rasterization to create the correct perspective division.

Question 2: What is Gimbal Lock? Why did you choose Quaternions over Euler angles for bone rotations, and how does SLERP operate?

Answer: Gimbal Lock is a mathematical phenomenon in Euler angles where rotation axes can align. This reduces the system's rotational degrees of freedom from three to two, causing sudden and unnatural joint snapping. We avoid this by using Quaternions ($\mathbb{H}$), which represent rotations as a 4D unit sphere vector $\mathbf{q} = [\cos(\theta/2) \quad \mathbf{u} \sin(\theta/2)]$. Quaternions are singularity-free, meaning they are completely immune to gimbal lock. Additionally, quaternions allow the use of Spherical Linear Interpolation (SLERP). While linear interpolation of Euler angles causes non-constant angular velocities, SLERP interpolates smoothly between orientations along the shortest path on the 4D unit sphere, resulting in fluid joint animations.

Question 3: Why is Tone Mapping necessary for HDR rendering? Why was ACES Filmic Tone Mapping chosen over a standard linear conversion, and what happens to exposure when transitioning to night mode?

Answer: High Dynamic Range (HDR) light values can exceed standard $[0, 1]$ display limits. Tone mapping scales these values from $[0, \infty)$ back to $[0, 1]$ so they can be shown on normal monitors. We chose ACES Filmic Tone Mapping because it simulates the exposure response of real film. Linear mapping simply clips bright pixels to white, resulting in flat, blown-out highlights. ACES, however, compresses high light intensities into a smooth curve, preserving rich details in sunset glares and street lamp halos. When switching to night mode, the environment's exposure value is lowered (`renderer.toneMappingExposure = 0.25`) to match nocturnal conditions, and the street lamps' intensity is increased to make them stand out in the dark.

Question 4: How does your Uniform Spatial Hash Grid optimize collision detection, and why is it superior to checking all bounding boxes in a single loop?

Answer: A standard loop checks every moving entity against all static colliders, scaling with a costly $\mathcal{O}(N \cdot M)$ complexity. Our Uniform Spatial Hash Grid solves this by dividing the scene into a 2D grid of cells. Static colliders are registered only in the specific grid cells they overlap. At runtime, when we check for player or bullet collisions, we use integer division of the coordinates by the cell size to determine the active cell in $\mathcal{O}(1)$ average time. We then evaluate collisions only against the few colliders inside that specific cell, keeping performance high and stable regardless of the scene's size.